

CSE 314 - Behavioral Design Patterns

General Process + Full Solutions

Part 1: General process for solving these exam problems (25-minute budget)

- **Read for keywords, not just the story (2 min).** 'switch between X at checkout/runtime' -> Strategy. 'goes through steps, invalid transitions forbidden' -> State. 'many components should auto-update when one thing changes' -> Observer. 'objects must not talk directly, a central object decides' -> Mediator. 'same overall steps for everyone, a few steps differ' -> Template Method.
- **Identify if it's a cross-pattern question (1 min).** If the story has two separate concerns (e.g. 'lifecycle with valid transitions' AND 'notify others on change'), you need two patterns working together, not one pattern stretched to cover both jobs.
- **Sketch the class skeleton on paper/comments first (3 min).** For State: an interface/abstract class per state, a context holding 'current state'. For Strategy: an interface per algorithm, a context holding a reference to the chosen strategy. For Observer: a Subject interface (attach/detach/notify) and an Observer interface (update). For Mediator: a Mediator interface plus concrete Colleague classes that only know the mediator. For Template Method: an abstract class with one final/concrete 'templateMethod()' calling abstract or hook steps.
- **Write the abstractions before the demo (10-12 min).** Get the interfaces and the context/mediator/subject compiling first; concrete classes are usually short and repetitive once the skeleton is right.
- **Write the main() demo last, matching exactly what the prompt's 'Task' line asks for (5 min).** Markers usually check the printed output against the described scenario, so cover every bullet point explicitly (e.g. an unsubscribe, an invalid transition, a queued request).
- **Sanity-check extensibility (2 min).** Ask: 'can I add a new state/strategy/observer/colleague without editing existing classes?' If not, you've hard-coded a decision (e.g. an if-else chain) somewhere that should be polymorphism instead.

Common mistakes to avoid

- Putting transition logic in the Context/State-holder class with a big switch statement -- defeats the purpose of the State pattern.
- Making the Subject (Observer pattern) call concrete display/observer methods directly instead of going through a common Observer interface.
- In Mediator questions, letting two colleague objects hold references to each other 'just this once' -- always route through the mediator.
- In Template Method questions, making the template method itself overridable (it should usually be 'final' to lock the skeleton) and forgetting to give optional steps a default/empty hook implementation.

Part 2: Full Java solutions

Each solution below corresponds to the matching problem in the practice-questions PDF.

S1 -- Smart Lock (State Pattern)

```
interface LockState {
    void enterPin(SmartLock ctx, int pin);
    void lockButton(SmartLock ctx);
    void forceUnlock(SmartLock ctx, String masterKey);
}

class LockedState implements LockState {
    public void enterPin(SmartLock ctx, int pin) {
        if (pin == ctx.correctPin) {
            System.out.println("PIN correct. Unlocking.");
            ctx.wrongAttempts = 0;
            ctx.setState(ctx.unlockedState);
        } else {
            ctx.wrongAttempts++;
            System.out.println("Wrong PIN (" + ctx.wrongAttempts + "/" + 3 + ").");
            if (ctx.wrongAttempts >= 3) {
                System.out.println("ALERT: possible intrusion! Lock jammed.");
                ctx.setState(ctx.jammedState);
            }
        }
    }
    public void lockButton(SmartLock ctx) { System.out.println("Already locked."); }
    public void forceUnlock(SmartLock ctx, String key) { System.out.println("Not jammed; use PIN instead."); }
}

class UnlockedState implements LockState {
    public void enterPin(SmartLock ctx, int pin) { System.out.println("Already unlocked."); }
    public void lockButton(SmartLock ctx) {
        System.out.println("Locking door.");
        ctx.setState(ctx.lockedState);
    }
    public void forceUnlock(SmartLock ctx, String key) { System.out.println("Already unlocked."); }
}

class JammedState implements LockState {
    public void enterPin(SmartLock ctx, int pin) { System.out.println("Lock is jammed!"); }
    public void lockButton(SmartLock ctx) { System.out.println("Lock is jammed!"); }
    public void forceUnlock(SmartLock ctx, String key) {
        if (key.equals("MASTER")) {
            System.out.println("Master key accepted. Unjammed and locked.");
            ctx.wrongAttempts = 0;
            ctx.setState(ctx.lockedState);
        } else {
            System.out.println("Wrong master key.");
        }
    }
}

class LowBatteryState implements LockState {
    public void enterPin(SmartLock ctx, int pin) { System.out.println("Low battery: please recharge."); }
    public void lockButton(SmartLock ctx) { System.out.println("Low battery: please recharge."); }
    public void forceUnlock(SmartLock ctx, String key) {
        if (key.equals("MASTER")) {
            System.out.println("Master key accepted despite low battery.");
            ctx.setState(ctx.lockedState);
        } else {
            System.out.println("Wrong master key.");
        }
    }
}

class SmartLock {
    LockState lockedState = new LockedState();
    LockState unlockedState = new UnlockedState();
    LockState jammedState = new JammedState();
    LockState lowBatteryState = new LowBatteryState();
}
```

```

    LockState current = lockedState;
    int correctPin = 1234;
    int wrongAttempts = 0;
    int battery = 100;

    void setState(LockState s) { current = s; }
    void enterPin(int pin) { current.enterPin(this, pin); }
    void lockButton() { current.lockButton(this); }
    void forceUnlock(String key) { current.forceUnlock(this, key); }

    void drainBattery(int amount) {
        battery -= amount;
        System.out.println("Battery now at " + battery + "%.");
        if (battery < 15 && current != jammedState) {
            System.out.println("Battery critical! Switching to LowBattery mode.");
            setState(lowBatteryState);
        }
    }
}

public class SmartLockDemo {
    public static void main(String[] args) {
        SmartLock lock = new SmartLock();
        lock.enterPin(9999);
        lock.enterPin(9999);
        lock.enterPin(9999); // -> jammed after 3 wrong tries
        lock.forceUnlock("WRONG");
        lock.forceUnlock("MASTER"); // -> back to locked
        lock.enterPin(1234); // -> unlocked
        lock.lockButton(); // -> locked
        lock.drainBattery(90); // -> low battery
        lock.enterPin(1234); // rejected, low battery
        lock.forceUnlock("MASTER"); // works even on low battery
    }
}

```

S2 -- Ride-Sharing (State + Observer)

```

import java.util.ArrayList;
import java.util.List;

// ---- Observer side ----
interface RideObserver { void update(Ride ride, String newState); }

class Rider implements RideObserver {
    public void update(Ride ride, String newState) {
        System.out.println("[Rider] Ride is now: " + newState);
    }
}

class Driver implements RideObserver {
    public void update(Ride ride, String newState) {
        System.out.println("[Driver] Ride is now: " + newState);
    }
}

// ---- State side ----
interface RideState {
    void accept(Ride ride);
    void startTrip(Ride ride);
    void complete(Ride ride);
    void cancel(Ride ride);
}

class RequestedState implements RideState {
    public void accept(Ride ride) { ride.setState(new AcceptedState(), "Accepted"); }
    public void startTrip(Ride ride) { System.out.println("Cannot start: not accepted yet."); }
    public void complete(Ride ride) { System.out.println("Cannot complete: ride not started."); }
    public void cancel(Ride ride) { ride.setState(new CancelledState(), "Cancelled"); }
}

class AcceptedState implements RideState {
    public void accept(Ride ride) { System.out.println("Already accepted."); }
    public void startTrip(Ride ride) { ride.setState(new OnTripState(), "OnTrip"); }
    public void complete(Ride ride) { System.out.println("Cannot complete: trip not started."); }
}

```

```

        public void cancel(Ride ride) { ride.setState(new CancelledState(), "Cancelled"); }
    }
    class OnTripState implements RideState {
        public void accept(Ride ride) { System.out.println("Already accepted."); }
        public void startTrip(Ride ride) { System.out.println("Already on trip."); }
        public void complete(Ride ride) { ride.setState(new CompletedState(), "Completed"); }
        public void cancel(Ride ride) { System.out.println("Cannot cancel: ride already in progress."); }
    }
    class CompletedState implements RideState {
        public void accept(Ride ride) { System.out.println("Ride already completed."); }
        public void startTrip(Ride ride) { System.out.println("Ride already completed."); }
        public void complete(Ride ride) { System.out.println("Already completed."); }
        public void cancel(Ride ride) { System.out.println("Cannot cancel a completed ride."); }
    }
    class CancelledState implements RideState {
        public void accept(Ride ride) { System.out.println("Ride is cancelled."); }
        public void startTrip(Ride ride) { System.out.println("Ride is cancelled."); }
        public void complete(Ride ride) { System.out.println("Ride is cancelled."); }
        public void cancel(Ride ride) { System.out.println("Already cancelled."); }
    }

    class Ride {
        private RideState state = new RequestedState();
        private final List<RideObserver> observers = new ArrayList<>();

        void addObserver(RideObserver o) { observers.add(o); }
        void removeObserver(RideObserver o) { observers.remove(o); }

        void setState(RideState s, String label) {
            this.state = s;
            for (RideObserver o : observers) o.update(this, label);
        }

        void accept() { state.accept(this); }
        void startTrip() { state.startTrip(this); }
        void complete() { state.complete(this); }
        void cancel() { state.cancel(this); }
    }

    public class RideDemo {
        public static void main(String[] args) {
            Ride ride1 = new Ride();
            Rider r1 = new Rider(); Driver d1 = new Driver();
            ride1.addObserver(r1); ride1.addObserver(d1);
            ride1.accept();
            ride1.startTrip();
            ride1.complete();

            System.out.println("---");
            Ride ride2 = new Ride();
            ride2.addObserver(new Rider());
            ride2.addObserver(new Driver());
            ride2.accept();
            ride2.cancel();
            ride2.startTrip(); // rejected: cancelled
        }
    }
}

```

ST1 -- Shipping Cost (Strategy)

```
interface ShippingStrategy {
    void computeCost(double distanceKm, double weightKg);
}

class StandardShipping implements ShippingStrategy {
    public void computeCost(double distanceKm, double weightKg) {
        double cost = 50 + 5 * distanceKm;
        System.out.println("Standard shipping cost: " + cost + " BDT");
    }
}

class ExpressShipping implements ShippingStrategy {
    public void computeCost(double distanceKm, double weightKg) {
        double cost = 100 + 12 * distanceKm;
        System.out.println("Express shipping cost: " + cost + " BDT");
    }
}

class SameDayShipping implements ShippingStrategy {
    public void computeCost(double distanceKm, double weightKg) {
        if (distanceKm > 30) {
            System.out.println("SameDay unavailable: distance exceeds 30 km.");
            return;
        }
        double cost = 150 + 20 * distanceKm;
        System.out.println("SameDay shipping cost: " + cost + " BDT");
    }
}

class Checkout {
    private ShippingStrategy strategy;
    void setStrategy(ShippingStrategy s) { this.strategy = s; }
    void checkout(double distanceKm, double weightKg) { strategy.computeCost(distanceKm, weightKg); }
}

public class ShippingDemo {
    public static void main(String[] args) {
        Checkout checkout = new Checkout();
        checkout.setStrategy(new StandardShipping());
        checkout.checkout(20, 3);
        checkout.setStrategy(new ExpressShipping());
        checkout.checkout(20, 3);
        checkout.setStrategy(new SameDayShipping());
        checkout.checkout(45, 3); // rejected
        checkout.checkout(10, 3); // accepted
    }
}
```

ST2 -- Document Compression (Strategy)

```
interface CompressionStrategy {
    int compress(String docName, int originalBytes);
}

class ZipCompression implements CompressionStrategy {
    public int compress(String docName, int originalBytes) {
        int saved = (int)(originalBytes * 0.6);
        System.out.println(docName + " saved with ZIP, saved " + saved + " bytes.");
        return saved;
    }
}

class RarCompression implements CompressionStrategy {
    public int compress(String docName, int originalBytes) {
        int saved = (int)(originalBytes * 0.7);
        System.out.println(docName + " saved with RAR, saved " + saved + " bytes.");
        return saved;
    }
}

class NoCompression implements CompressionStrategy {
    public int compress(String docName, int originalBytes) {
        System.out.println(docName + " saved with NO compression, saved 0 bytes.");
        return 0;
    }
}
```

```

    }
}

class SaveManager {
    private CompressionStrategy strategy;
    void setStrategy(CompressionStrategy s) { this.strategy = s; }
    void save(String docName, int originalBytes) { strategy.compress(docName, originalBytes); }
}

public class CompressionDemo {
    public static void main(String[] args) {
        SaveManager mgr = new SaveManager();
        mgr.setStrategy(new ZipCompression());
        mgr.save("report.docx", 1000);
        mgr.setStrategy(new RarCompression());
        mgr.save("report.docx", 1000);
        mgr.setStrategy(new NoCompression());
        mgr.save("report.docx", 1000);
    }
}

```

O1 -- Weather Station (Observer)

```
import java.util.ArrayList;
import java.util.List;

interface Display {
    void update(double temp, double humidity);
}

class CurrentConditionsDisplay implements Display {
    public void update(double temp, double humidity) {
        System.out.println("[Current] Temp: " + temp + "C, Humidity: " + humidity + "%");
    }
}

class StatisticsDisplay implements Display {
    private double min = Double.MAX_VALUE, max = Double.MIN_VALUE, sum = 0;
    private int count = 0;
    public void update(double temp, double humidity) {
        min = Math.min(min, temp); max = Math.max(max, temp);
        sum += temp; count++;
        System.out.println("[Stats] min=" + min + " max=" + max + " avg=" + (sum / count));
    }
}

class AlertDisplay implements Display {
    public void update(double temp, double humidity) {
        if (temp > 40) System.out.println("[Alert] Heat warning! Temp = " + temp);
    }
}

class WeatherStation {
    private final List<Display> displays = new ArrayList<>();
    void subscribe(Display d) { displays.add(d); }
    void unsubscribe(Display d) { displays.remove(d); }
    void setMeasurements(double temp, double humidity) {
        for (Display d : displays) d.update(temp, humidity);
    }
}

public class WeatherDemo {
    public static void main(String[] args) {
        WeatherStation station = new WeatherStation();
        Display current = new CurrentConditionsDisplay();
        Display stats = new StatisticsDisplay();
        Display alert = new AlertDisplay();
        station.subscribe(current);
        station.subscribe(stats);
        station.subscribe(alert);

        station.setMeasurements(30, 65);
        station.setMeasurements(42, 50);
        station.unsubscribe(alert);
        station.setMeasurements(45, 40); // alert no longer fires
        station.setMeasurements(28, 70);
    }
}
```

O2 -- Crypto Price Alerts (Observer + Strategy)

```
import java.util.ArrayList;
import java.util.List;

interface AlertStrategy {
    boolean shouldFire(double lastPrice, double newPrice);
    String describe();
}

class AboveThresholdStrategy implements AlertStrategy {
    private final double threshold;
    AboveThresholdStrategy(double t) { threshold = t; }
    public boolean shouldFire(double lastPrice, double newPrice) { return newPrice > threshold; }
    public String describe() { return "price above " + threshold; }
}
```

```

    }
    class PercentDropStrategy implements AlertStrategy {
        private final double percent;
        PercentDropStrategy(double p) { percent = p; }
        public boolean shouldFire(double lastPrice, double newPrice) {
            return lastPrice > 0 && (lastPrice - newPrice) / lastPrice * 100 >= percent;
        }
        public String describe() { return "drop of " + percent + "%"; }
    }

    interface Subscriber { void onPriceChange(String coin, double lastPrice, double newPrice); }

    class User implements Subscriber {
        private final String name;
        private final AlertStrategy strategy;
        User(String name, AlertStrategy strategy) { this.name = name; this.strategy = strategy; }
        public void onPriceChange(String coin, double lastPrice, double newPrice) {
            if (strategy.shouldFire(lastPrice, newPrice)) {
                System.out.println(name + " ALERT on " + coin + " (" + strategy.describe() + "): " + newPrice);
            }
        }
    }

    class CoinFeed {
        private final String coinName;
        private double price;
        private final List<Subscriber> subs = new ArrayList<>();
        CoinFeed(String name, double initialPrice) { coinName = name; price = initialPrice; }
        void subscribe(Subscriber s) { subs.add(s); }
        void unsubscribe(Subscriber s) { subs.remove(s); }
        void setPrice(double newPrice) {
            double old = price;
            price = newPrice;
            for (Subscriber s : subs) s.onPriceChange(coinName, old, newPrice);
        }
    }

    public class CryptoAlertDemo {
        public static void main(String[] args) {
            CoinFeed btc = new CoinFeed("BTC", 60000);
            User alice = new User("Alice", new AboveThresholdStrategy(65000));
            User bob = new User("Bob", new PercentDropStrategy(5));
            btc.subscribe(alice);
            btc.subscribe(bob);

            btc.setPrice(66000); // Alice fires
            btc.setPrice(62000); // ~6% drop -> Bob fires
            btc.unsubscribe(bob);
            btc.setPrice(58000); // Bob no longer notified
        }
    }

```

M1 -- Air Traffic Control (Mediator)

```
import java.util.LinkedList;
import java.util.Queue;

interface Tower {
    void requestLanding(Flight f);
    void requestTakeoff(Flight f);
    void notifyDone(Flight f);
}

class ControlTower implements Tower {
    private boolean runwayBusy = false;
    private final Queue<Flight> waiting = new LinkedList<>();

    public void requestLanding(Flight f) { handleRequest(f, "land"); }
    public void requestTakeoff(Flight f) { handleRequest(f, "take off"); }

    private void handleRequest(Flight f, String action) {
        if (!runwayBusy) {
            runwayBusy = true;
            System.out.println(f.name + " cleared to " + action + ".");
        } else {
            System.out.println(f.name + " must HOLD, runway busy.");
            waiting.add(f);
        }
    }

    public void notifyDone(Flight f) {
        System.out.println(f.name + " finished using the runway.");
        runwayBusy = false;
        if (!waiting.isEmpty()) {
            Flight next = waiting.poll();
            runwayBusy = true;
            System.out.println(next.name + " now cleared (was waiting).");
        }
    }
}

class Flight {
    final String name;
    final Tower tower;
    Flight(String name, Tower tower) { this.name = name; this.tower = tower; }
    void land() { tower.requestLanding(this); }
    void takeoff() { tower.requestTakeoff(this); }
    void finish() { tower.notifyDone(this); }
}

public class AtcDemo {
    public static void main(String[] args) {
        ControlTower tower = new ControlTower();
        Flight f101 = new Flight("Flight101", tower);
        Flight f202 = new Flight("Flight202", tower);
        Flight f303 = new Flight("Flight303", tower);

        f101.land();
        f202.takeoff(); // must hold
        f303.land(); // must hold
        f101.finish(); // frees runway -> f202 cleared
        f202.finish(); // frees runway -> f303 cleared
        f303.finish();
    }
}
```

M2 -- Elevator System (Mediator + State)

```
import java.util.LinkedList;
import java.util.Queue;

interface ElevatorState {
    void moveTo(Elevator ctx, int floor);
    void arrive(Elevator ctx);
}
```

```

}
class IdleState implements ElevatorState {
    public void moveTo(Elevator ctx, int floor) {
        System.out.println(ctx.name + " moving from " + ctx.currentFloor + " to " + floor);
        ctx.targetFloor = floor;
        ctx.state = floor > ctx.currentFloor ? ctx.movingUp : ctx.movingDown;
    }
    public void arrive(Elevator ctx) { System.out.println(ctx.name + " already idle."); }
}
class MovingState implements ElevatorState {
    public void moveTo(Elevator ctx, int floor) { System.out.println(ctx.name + " busy, cannot retarget now."); }
    public void arrive(Elevator ctx) {
        ctx.currentFloor = ctx.targetFloor;
        System.out.println(ctx.name + " arrived at floor " + ctx.currentFloor + ", doors open.");
        ctx.state = ctx.doorsOpen;
    }
}
class DoorsOpenState implements ElevatorState {
    public void moveTo(Elevator ctx, int floor) { System.out.println(ctx.name + " must close doors first."); }
    public void arrive(Elevator ctx) { System.out.println(ctx.name + " already at doors open."); }
    void closeDoors(Elevator ctx) {
        System.out.println(ctx.name + " doors closed, now idle.");
        ctx.state = ctx.idle;
    }
}

class Elevator {
    final String name;
    int currentFloor = 0, targetFloor = 0;
    ElevatorState idle = new IdleState();
    ElevatorState movingUp = new MovingState();
    ElevatorState movingDown = new MovingState();
    DoorsOpenState doorsOpen = new DoorsOpenState();
    ElevatorState state = idle;

    Elevator(String name) { this.name = name; }
    boolean isIdle() { return state == idle; }
    void moveTo(int floor) { state.moveTo(this, floor); }
    void simulateArrival() { state.arrive(this); }
    void closeDoors() { if (state == doorsOpen) doorsOpen.closeDoors(this); }
}

class ElevatorController {
    private final Elevator[] elevators;
    private final Queue<Integer> pending = new LinkedList<>();
    ElevatorController(Elevator[] elevators) { this.elevators = elevators; }

    void requestFloor(int floor) {
        Elevator chosen = null;
        for (Elevator e : elevators) {
            if (e.isIdle() && (chosen == null ||
                Math.abs(e.currentFloor - floor) < Math.abs(chosen.currentFloor - floor))) {
                chosen = e;
            }
        }
        if (chosen != null) {
            chosen.moveTo(floor);
        } else {
            System.out.println("All elevators busy, queuing floor " + floor);
            pending.add(floor);
        }
    }

    void elevatorBecameIdle() {
        if (!pending.isEmpty()) {
            int floor = pending.poll();
            System.out.println("Dispatching queued request for floor " + floor);
            requestFloor(floor);
        }
    }
}

public class ElevatorDemo {

```

```
public static void main(String[] args) {
    Elevator e1 = new Elevator("Elevator-A");
    Elevator e2 = new Elevator("Elevator-B");
    ElevatorController controller = new ElevatorController(new Elevator[]{e1, e2});

    controller.requestFloor(5);    // e1 or e2 dispatched
    controller.requestFloor(2);    // the other dispatched
    controller.requestFloor(8);    // both busy -> queued

    e1.simulateArrival();
    e1.closeDoors();
    controller.elevatorBecameIdle(); // dispatches queued floor 8
}
}
```

T1 -- Report Export (Template Method)

```
import java.util.List;

abstract class ReportExporter {
    // template method: fixed skeleton
    final void export(String title, List<String> rows) {
        List<String> data = gatherData(rows);
        applyHeader(title);
        writeBody(data);
        applyFooter(); // optional hook, default no-op
        finalizeFile();
    }

    private List<String> gatherData(List<String> rows) {
        System.out.println("Gathering " + rows.size() + " rows of data...");
        return rows;
    }

    protected abstract void applyHeader(String title);
    protected abstract void writeBody(List<String> data);
    protected void applyFooter() { /* optional: default does nothing */ }

    private void finalizeFile() {
        System.out.println("File finalized and saved.\n");
    }
}

class PdfReportExporter extends ReportExporter {
    protected void applyHeader(String title) {
        System.out.println("=== " + title + " (PDF) ===");
    }
    protected void writeBody(List<String> data) {
        for (int i = 0; i < data.size(); i++) {
            System.out.println("Row: " + data.get(i));
            if ((i + 1) % 3 == 0) System.out.println("--- page break ---");
        }
    }
    protected void applyFooter() {
        System.out.println("Page footer: generated report");
    }
}

class CsvReportExporter extends ReportExporter {
    protected void applyHeader(String title) {
        System.out.println("id,value (CSV: " + title + ")");
    }
    protected void writeBody(List<String> data) {
        for (String row : data) System.out.println(row.replace(" ", ","));
    }
    // footer intentionally not overridden -> uses default no-op
}

public class ReportDemo {
    public static void main(String[] args) {
        List<String> rows = List.of("1 apple", "2 banana", "3 cherry", "4 date", "5 fig");
        new PdfReportExporter().export("Sales Report", rows);
        new CsvReportExporter().export("Sales Report", rows);
    }
}
```

T2 -- Data Import Pipeline (Template Method + Strategy)

```
import java.util.ArrayList;
import java.util.List;

class Record {
    String id, name;
    Record(String id, String name) { this.id = id; this.name = name; }
    public String toString() { return "Record(" + id + ", " + name + ")"; }
}
```

```

// ---- Strategy: pluggable sort step ----
interface SortStrategy { void sort(List<Record> records); }

class SortByIdStrategy implements SortStrategy {
    public void sort(List<Record> records) {
        records.sort((a, b) -> a.id.compareTo(b.id));
    }
}
class SortByNameStrategy implements SortStrategy {
    public void sort(List<Record> records) {
        records.sort((a, b) -> a.name.compareTo(b.name));
    }
}

// ---- Template Method: fixed pipeline skeleton ----
abstract class ImportPipeline {
    private SortStrategy sortStrategy;
    void setSortStrategy(SortStrategy s) { this.sortStrategy = s; }

    final void run(List<String> rawLines) {
        List<Record> parsed = readRecords(rawLines);
        List<Record> valid = new ArrayList<>();
        int invalidCount = 0;
        for (Record r : parsed) {
            if (validate(r)) valid.add(r); else invalidCount++;
        }
        sortStrategy.sort(valid);
        printSummary(valid, invalidCount);
    }

    protected abstract List<Record> readRecords(List<String> rawLines);
    protected abstract boolean validate(Record r);

    private void printSummary(List<Record> valid, int invalidCount) {
        System.out.println("Imported " + valid.size() + " valid, skipped " + invalidCount + " invalid.");
        for (Record r : valid) System.out.println("  " + r);
        System.out.println();
    }
}

class CsvSource extends ImportPipeline {
    protected List<Record> readRecords(List<String> rawLines) {
        List<Record> out = new ArrayList<>();
        for (String line : rawLines) {
            String[] fields = line.split(",");
            // simulate: valid CSV record needs exactly 3 fields, else keep as-is (id may be blank)
            if (fields.length == 3) out.add(new Record(fields[0], fields[1]));
            else out.add(new Record("", ""));
        }
        return out;
    }
    protected boolean validate(Record r) { return !r.id.isEmpty(); }
}

class JsonSource extends ImportPipeline {
    protected List<Record> readRecords(List<String> rawLines) {
        List<Record> out = new ArrayList<>();
        for (String line : rawLines) {
            // simplistic simulated parse: "id:1,name:Bob" or missing id -> null
            String id = line.contains("id:") ? line.split("id:")[1].split(",")[0] : null;
            String name = line.contains("name:") ? line.split("name:")[1] : "unknown";
            out.add(new Record(id, name));
        }
        return out;
    }
    protected boolean validate(Record r) { return r.id != null; }
}

public class ImportDemo {
    public static void main(String[] args) {
        List<String> csvLines = List.of("1,Zack,x", "2,Amy,y", "bad,line");
        CsvSource csv = new CsvSource();
        csv.setSortStrategy(new SortByNameStrategy());
    }
}

```

```
        csv.run(csvLines);

        List<String> jsonLines = List.of("id:3,name:Mona", "name:NoId", "id:1,name:Anil");
        JsonSource json = new JsonSource();
        json.setSortStrategy(new SortByIdStrategy());
        json.run(jsonLines);
    }
}
```